

PCCP 700+ Roadmap

Lộ trình thực chiến 37 ngày bằng JavaScript

30/07 → 04/09

37 ngày học có lịch R1/R3/R7

48 bài

29 bắt buộc · 14 nên làm · 5 tùy chọn

05/09/2026

Ngày thi do người học cung cấp

5 mock

2 synthetic · 2 chính thức · 1 tổng duyệt

MỤC TIÊU VẬN HÀNH Viết chắc các dạng phổ biến, giảm lỗi JavaScript/hidden test và quản lý 120 phút. Kế hoạch tối ưu xác suất đạt 700+, không bảo đảm điểm.

Nghiên cứu và tổng hợp nguồn chính thức/cộng đồng · 30/07/2026

Mục lục

Bản Word này là tài liệu tổng hợp. Lịch ngày, tracker CSV, Error Log, bộ template và cheat sheet nằm trong cùng thư mục.

- Phần 1** PCCP và ý nghĩa của mục tiêu 700+
 - Phần 2** Đánh giá trình độ hiện tại
 - Phần 3** Bản đồ kiến thức theo mục tiêu 700+
 - Phần 4** Bộ template JavaScript tối thiểu
 - Phần 5** Nhận diện dạng bài
 - Phần 6** Lộ trình 37 ngày
 - Phần 7** Khóa học và bộ luyện phù hợp trên Programmers
 - Phần 8** Danh sách 48 bài luyện
 - Phần 9** Quy trình học một bài
 - Phần 10** Error Log
 - Phần 11** Hidden test checklist
 - Phần 12** Chiến thuật thi 120 phút
 - Phần 13** Mock test
 - Phần 14** Dashboard tiến độ
 - Phần 15** Cheat sheet cuối
- Nguồn và giới hạn**
-

Bắt đầu trong hôm nay

- 1. Mở [PCCP_37_Day_Daily_Plan.md](#), làm đúng D1 trong 180 phút.
- 2. Mở [PCCP_Problem_Tracker.csv](#), lọc `planned_first_attempt = 2026-07-30`; làm P25.
- 3. Viết lại 4 template đầu trong [PCCP_JavaScript_Templates.md](#), đặc biệt matrix và numeric sort.
- 4. Ghi lỗi thật vào [PCCP_Error_Log_Template.csv](#), không chỉ ghi "sai".
- 5. Chưa bấm link/đọc đề P09–P16: đó là hai bộ mock chính thức được giữ mới đến 29/08 và 31/08. Tên/pattern vẫn xuất hiện trong tracker để quản lý phạm vi.

Bộ tài liệu

File	Dùng khi nào
PCCP_37_Day_Daily_Plan.md	Mở đầu mỗi ngày; lịch đủ 37 ngày, review R1/R3/R7 và 5 mock
PCCP_JavaScript_Templates.md	Học/recode template; mỗi template có dấu hiệu, code, lỗi, ví dụ, checklist
PCCP_Problem_Tracker.csv	48 bài thật, link, nhóm, Level, pattern, timebox, hidden risk, lịch review
PCCP_Error_Log_Template.csv	Root cause, revealing test, rule và ngày làm lại
PCCP_Final_Cheat_Sheet.md	Bản ôn 4 trang cho 48 giờ cuối
sources.md	85 URL đã rà soát, phân loại chính thức/kỹ thuật/cộng đồng

Phần 1 — PCCP và ý nghĩa của mục tiêu 700+

1.1 Dữ kiện chính thức

Hạng mục	Kết quả kiểm chứng
Hình thức	4 câu viết code
Thời gian	120 phút
Tổng điểm	1.000
Ngôn ngữ	Python, JavaScript, Java, C, C++, C#; chọn một
Điểm đỗ	Từ 400
Mục tiêu 700	Thuộc LV.3, vì LV.3 là 600–749; LV.4 bắt đầu ở 750
Năng lực	Hiện thực chương trình đúng yêu cầu; chọn cấu trúc dữ liệu/thuật toán phù hợp; viết đúng và hiệu quả
Phạm vi ví dụ	String, Array, Greedy, Sort; Stack, Queue, Deque, Hash, Binary Search, DFS, BFS; Graph, Tree, Heap, DP

Nguồn chính: [trang PCCP](#), [phạm vi PCCP](#), [PDF giới thiệu chứng chỉ](#).

Một số câu mô tả cũ trên các trang chính thức vẫn còn ghi "4 ngôn ngữ", nhưng danh sách môn thi hiện hành và PDF chính thức cùng liệt kê **6**; JavaScript xuất hiện nhất quán trong cả hai, nên kế hoạch dùng JavaScript không bị ảnh hưởng.

Các mốc cấp độ

Cấp độ	Điểm
LV.1	400–499
LV.2	500–599
LV.3	600–749
LV.4	750–899
LV.5	900–1.000

Vì vậy, **700 không đồng nghĩa LV.4**. Mục tiêu chính xác là nằm chắc trong vùng LV.3 cao.

1.2 Điều chưa được công bố

Không tìm thấy nguồn chính thức hiện hành cho:

- trọng số cố định của từng câu;
- công thức quy đổi từng test case thành điểm;
- cơ chế partial score chi tiết;
- khẳng định “ba câu đầu bằng 700/750/800”;
- phiên bản Node.js cụ thể dùng cho mọi lượt thi.

Do đó, tài liệu **không dùng** tin đồn 200/300/300/200. Một số trải nghiệm cộng đồng nói rằng sample/visible test không phản ánh hết hidden/efficiency test; điều này chỉ được dùng để tăng thời gian audit, không dùng để suy ra cách chấm. Xem phân loại nguồn tại sources.md.

1.3 Mục tiêu vận hành thay cho “công thức điểm”

Chiến lược luyện là:

1. Nhận diện nhanh và hoàn thành trọn vẹn nhiều bài khả thi nhất.
2. Trong phòng thi, ưu tiên ba lời giải có xác suất đúng cao.
3. Khi ba bài đã ổn, tiến thêm ở bài còn lại nếu có hướng rõ hoặc test case khả thi.
4. Không hy sinh hai bài dễ cho một bài khó.
5. Dành 15–18 phút cuối cho hidden test, complexity và submit.

“Ba bài chắc + tiến thêm ở bài còn lại” là **heuristic quản lý rủi ro**, không phải bảo đảm 700 điểm.

1.4 Lưu ý về ngày thi

[Lịch tryout công khai](#) và [API lịch công khai](#) không hiển thị kỳ thi PCCP thường lệ ngày 05/09/2026 tại thời điểm 30/07/2026. Kế hoạch vẫn giữ ngày người học cung cấp; đây có thể là lịch riêng hoặc lượt chưa công bố. Hãy xác nhận thư mời, giờ, múi giờ và hướng dẫn submit của chính lượt thi.

Phần 2 — Đánh giá trình độ hiện tại

2.1 Chẩn đoán

Ví dụ `arr1[row, col]`, vòng cột dùng `arr1.length` và phải khởi tạo thủ công `answer[row]` cho thấy người học **đã có ý định thuật toán đúng** — duyệt từng ô và cộng — nhưng mô hình dữ liệu và cú pháp chưa tự động hóa.

Điểm nghẽn chính không phải “không có tư duy”, mà là bốn tầng chuyển đổi:

yêu cầu → invariant → pseudocode → JavaScript đúng biên

Nếu tập trung, lỗi cú pháp, khởi tạo, sort, matrix, `Map/Set` có thể giảm mạnh trong 7–14 ngày. Nhận diện pattern, complexity, state và hidden test phải được luyện lặp suốt 37 ngày.

2.2 Bảng đánh giá

Điểm mạnh	Điểm yếu	Rủi ro thi	Cách xử lý	Ưu tiên
Là lập trình viên, hiểu control flow	Viết JS thuật toán chưa trôi	Mất 10–20 phút cho cú pháp	20 phút recall + recode từ file trống mỗi ngày	Rất cao
Đã tiếp xúc Big O, array, hash, Set	Kiến thức rời, chưa có cue nhận diện	Biết tên pattern nhưng chọn sai	Checklist 60 giây + nói dấu hiệu trước khi code	Rất cao
Logic cơ bản không phải nút thắt lớn nhất	Matrix/index/boundary yếu	Sample vuông pass, hidden matrix chữ nhật fail	Drill 1×1 , $1 \times N$, $N \times 1$, 2×3 ; Error Log bắt buộc	Rất cao

Điểm mạnh	Điểm yếu	Rủi ro thi	Cách xử lý	Ưu tiên
Đã biết ý tưởng frequency/two pointer	Khó chuyển ý tưởng thành invariant/code	Code chấp vá, state update sai thứ tự	Pseudocode 5–10 dòng; một câu invariant trước code	Rất cao
Có khái niệm heap/prefix	Chưa tự cài template	Không kịp code trong thi	Thuộc API và class heap duy nhất; không học nhiều biến thể	Cao
Có thể debug khi thấy lỗi	Ít kinh nghiệm hidden test	Pass sample nhưng fail biên/efficiency	Tự tạo revealing test trước submit; 15–18 phút audit	Rất cao
Có 37 ngày, đủ tạo vòng lặp sửa lỗi	Lo ngợp 30–40 thuật toán	Học lan man, không có fluency	Ba tầng kiến thức; khóa 10–12 pattern cốt lõi	Rất cao

2.3 Phân biệt ba loại lỗi

Loại	Dấu hiệu	Ví dụ	Cách sửa
JavaScript	Ý tưởng đúng nhưng code sai/ngheñ	<code>arr[row, col]</code> , sort thiếu comparator	Template + drill cực nhỏ + recode
Implementation	Pattern đúng nhưng state/invariant sai	đánh <code>visited</code> quá muộn; quên reset	Trace bảng trạng thái; test tối thiểu làm lộ lỗi
Thuật toán	Complexity hoặc pattern không phù hợp	<code>indexOf</code> trong loop lớn; DFS cho shortest path	Quay lại constraints, brute force và chứng minh

Phần 3 — Bản đồ kiến thức theo mục tiêu 700+

3.1 Cách đọc

- **Phải thuộc:** viết được không nhìn.
- **Cốt lõi:** hiểu, nhận diện và tự code được bài chuẩn.
- **Cơ bản:** nhận diện, dùng template khi cần; không đào sâu biến thể.
- **Bỏ qua:** không đầu tư trong 37 ngày, trừ khi Error Log/mocks chỉ ra nhu cầu.
- Số bài là mức tối thiểu định hướng và **có chồng lấp**; một bài có thể tính cho nhiều chủ đề.

3.2 Knowledge map

Chủ đề	Dùng để giải quyết	Dấu hiệu nhận diện	Mức	Khó	Bài tối thiểu	Lỗi JavaScript thường gặp
JavaScript cho coding test	Chuyển ý tưởng thành code	Mọi bài	Phải thuộc	1/5	Drill hằng ngày	scope, mutation, undefined, nhầm API
Complexity	Loại bỏ lời giải quá chậm	Constraints lớn	Phải thuộc	2/5	Mọi bài	Bỏ sót <code>indexOf/includes/sort</code> trong loop
Array	Lưu/duyệt sequence	Danh sách, index, aggregate	Phải thuộc	1/5	8	off-by-one, mutate input
String	Parse, compare, tokenize	Ký tự, format, prefix	Cốt lõi	2/5	5	string immutable, lexicographic compare
Matrix/Grid	Trạng thái 2D	Hàng/cột, tọa độ, bản đồ	Phải thuộc	2/5	5	<code>[row,col]</code> , dùng sai số cột, shared rows
Simulation/Implementation	Thực thi rule theo thứ tự	“Mỗi giây/lệnh/lượt”, nhiều điều kiện	Cốt lõi	2–3/5	8	update sai thứ tự, quên reset/clamp
Hash/Map	Tra cứu key→value nhanh	Đếm, ghép, lookup theo tên/id	Phải thuộc	2/5	6	<code>get</code> trả <code>undefined</code> , key coercion

Chủ đề	Dùng để giải quyết	Dấu hiệu nhận diện	Mức	Khó	Bài tối thiểu	Lỗi JavaScript thường gặp
Set	Membership/unique	Tồn tại, loại trùng	Phải thuộc	1/5	4	dùng array <code>includes</code> trong loop lớn
Frequency Counter	Đếm tần suất	anagram, multiset, loại/số lượng	Phải thuộc	2/5	4	thiếu <code>?? 0</code> , không giảm/xóa đúng
Sort	Chuẩn hóa thứ tự, greedy	"nhỏ nhất", interval, comparator	Phải thuộc	2/5	6	sort chuỗi mặc định, mutate, comparator sai
Two Pointer	Thu hẹp hai đầu/đoạn	Array sort, cặp, sum, segment	Cơ bản	3/5	3	pointer đứng yên; dùng invariant tổng đơn điệu với số âm mà chưa chứng minh
Sliding Window	Đoạn liên tiếp cập nhật dần	cửa sổ độ dài K/điều kiện	Cơ bản	3/5	3	bỏ sót window đầu/cuối
Prefix Sum	Truy vấn/tính tổng đoạn	nhiều range sum, cumulative	Cơ bản	2/5	3	lệch <code>right+1</code> , overflow
Stack	LIFO, undo, nearest	ngoặc, pop điều kiện, đường hiện tại	Phải thuộc	2/5	4	pop rỗng, nhầm top
Queue	FIFO, event, BFS	thứ tự đến, hàng chờ	Phải thuộc	2/5	5	<code>shift()</code> lặp, head/length sai
Deque	Hai đầu	push/pop ở cả hai đầu	Cơ bản	3/5	1-2	giả lập bằng <code>unshift/shift</code> trên dữ liệu lớn
Heap/Priority Queue	Lấy min/max liên tục	scheduling, top priority, Dijkstra	Cốt lõi	4/5	3	comparator đảo, bubble sai, pop size 1
Binary Search	Tìm value/cận/answer đơn điệu	"nhỏ nhất thỏa", giới hạn lớn	Cốt lõi	3/5	4	loop vô hạn, cận sai, predicate không đơn điệu
Brute Force	Liệt kê khi input nhỏ	N nhỏ, thử mọi cặp/cấu hình	Cốt lõi	2/5	4	không tính search space trước
Recursion	Diễn tả cây lựa chọn	subtree, choose/unchoose	Cơ bản	3/5	2	thiếu base case, call stack sâu
Backtracking	Thử và hoàn tác	chọn thứ tự/tập hợp, constraints nhỏ	Cơ bản	4/5	3	quên undo/reset; shared state
Combination	Chọn không quan tâm thứ tự	chọn K trong N	Cơ bản	3/5	2	lặp hoán vị thừa, start index sai
Permutation	Chọn có thứ tự	mọi lịch/thứ tự	Cơ bản	4/5	2	visited không reset, factorial explosion
DFS	Vùng liên thông, exhaustive traversal	"connected", đi sâu	Cốt lõi	3/5	5	recursion sâu, visited sai lúc
BFS	Shortest path không trọng số	ít bước nhất trên grid/graph	Cốt lõi	3/5	6	mark visited khi dequeue, queue chậm
Graph	Quan hệ node-edge	mạng, đường nối, dependency	Cốt lõi	3/5	5	sai directed/undirected, index 0/1
Tree	Phân cấp không chu trình	parent/child, subtree	Cơ bản	3/5	2	quay về parent vô hạn
Greedy	Quyết định cục bộ có chứng minh	interval, sort rồi chọn	Cốt lõi	3/5	5	"trông hợp lý" nhưng không có exchange proof
Dynamic Programming	State lặp và tối ưu/đếm	cùng subproblem, min/max/count	Cốt lõi	4/5	5	state mơ hồ, base/order sai
Union Find	Thành phần động/kết nối	union/query connectivity	Cơ bản	3/5	1	thiếu path compression/rank; index
Dijkstra	Shortest path trọng số không âm	graph có weight/cost	Cơ bản	4/5	1-2	stale heap entry, dùng khi có cạnh âm
Trie	Prefix lượng lớn	nhiều insert/query prefix	Bỏ qua	4/5	0	object node/state phức tạp
Segment/Fenwick tree	Truy vấn/update range động	nhiều update + query	Bỏ qua	5/5	0	index 1-based, code dài

Chủ đề	Dùng để giải quyết	Dấu hiệu nhận diện	Mức	Khó	Bài tối thiểu	Lỗi JavaScript thường gặp
Graph/DP nâng cao	Bài chuyên sâu	state nhiều chiều/chứng minh nặng	Bỏ qua	5/5	0	dễ phá lịch, ít ROI cho mục tiêu

Kết luận phạm vi

Không có yêu cầu phải học hết bảng. Cần khóa:

- 8 mẫu cú pháp nền tảng;
- 10–12 pattern cốt lõi;
- các pattern bổ sung chỉ khi bài trong lịch cần;
- bỏ Trie/segment tree/Fenwick/DP nâng cao trong kế hoạch chính.

Phần 4 — Bộ template JavaScript tối thiểu

Toàn bộ code và giải thích nằm tại [PCCP JavaScript Templates.md](#). Bộ này có 22 template, nhưng không phải 22 thứ phải thuộc ngay.

4.1 Ba tầng pattern và template

Tầng 1 — Phải viết được không nhìn, trước 02/08

1. Duyệt array.
2. Duyệt matrix chữ nhật.
3. Tạo matrix/hàng kết quả độc lập.
4. `Map` frequency.
5. `Set` membership.
6. Numeric sort.
7. Stack.
8. Queue bằng head index.

Tầng 2 — Cốt lõi, khóa dần trước 23/08

1. Simulation skeleton/state order.
2. BFS.
3. Iterative DFS.
4. Binary search exact.
5. `firstTrue`/binary search on answer.
6. Min-heap/Priority Queue.
7. Adjacency list.
8. Brute force/backtracking skeleton.
9. Greedy sau sort.
10. DP 1D.
11. DP 2D cơ bản.
12. Prefix/two pointer/sliding window ở mức nhận diện và code chuẩn.

Tầng 3 — Chỉ tra khi bài thật cần

Dijkstra, Union Find, combination/permutation biến thể. Không đầu tư Trie, segment tree, Fenwick tree hay DP phức tạp trước mục tiêu hiện tại.

4.2 Cảnh báo JavaScript bắt buộc

- `Array(n).fill([])` làm các phần tử cùng trỏ tới một array.
- Tạo matrix: `Array.from({ length: rows }, () => Array(cols).fill(0))`.

- `sort()` mặc định sort theo chuỗi và mutate array.
- `Map.get(key) ?? 0` chỉ fallback khi giá trị là `null/undefined`; khác `|| 0` ở chỗ không ghi đè số `0` hợp lệ. Muốn phân biệt key vắng, dùng `map.has(key)`.
- Queue lớn dùng `head`, tránh `shift()` liên tục.
- Recursion depth phụ thuộc runtime; graph/path sâu nên dùng iterative DFS.
- Spread/slice là shallow copy; nested object vẫn dùng chung reference.
- `Number.MAX_SAFE_INTEGER` là biên số nguyên an toàn; dùng `BigInt` chỉ khi thật sự cần và không trộn hai kiểu.
- Closure/biến ngoài `solution` có thể giữ state ngoài ý muốn; tạo state bên trong mỗi lần gọi.
- Không dùng bitwise để "tối ưu" số lớn: toán tử bitwise ép về số nguyên 32-bit.

Phần 5 — Nhận diện dạng bài

5.1 Decision tree

Đọc constraints và output

Rule chạy theo giây/lệnh/luot?

Cần đếm/tồn tại/ghép theo key?

Đoạn liên tiếp?

Liên tục lấy nhỏ nhất/lớn nhất?

Grid/graph?

Tìm min/max thỏa predicate đơn điệu?

Input nhỏ, thử mọi lựa chọn?

Quyết định cục bộ có thể chứng minh?

State lặp, cần min/max/số cách?

→ Simulation

→ Map / Set

Tổng nhiều query

Cửa sổ cố định/tiến một chiều

Mảng sort/hai đầu

→ Prefix Sum

→ Sliding Window

→ Two Pointer

→ Heap

Đường ít bước nhất, không trọng số

Vùng liên thông/duyệt hết

Cạnh có trọng số không âm

→ BFS

→ DFS hoặc BFS

→ Dijkstra

→ Binary Search on answer

→ Brute Force/Backtracking

→ Greedy

→ DP

5.2 Checklist 60 giây

1. Input lớn nhất bao nhiêu? Có nhiều test không?
2. $O(N^2)$ chạy được không? Có loop ẩn?
3. Đây là mô phỏng hay bài tối ưu?
4. Cần lưu `visited`/state nào?
5. Predicate có tính đơn điệu không?
6. Subproblem nào bị tính lặp?
7. Output là giá trị, số cách hay đường đi?
8. Search space brute force là bao nhiêu?
9. Có tie-break hoặc "không có đáp án" không?
10. `Number` có an toàn không?

Nếu sau 60 giây vẫn chưa biết pattern, viết brute force và complexity trước; tên thuật toán sẽ rõ hơn từ bottleneck.

Phần 6 — Lộ trình 37 ngày

Lịch chi tiết đủ từng ngày nằm tại [PCCP 37 Day Daily Plan.md](#).

Giai đoạn	Ngày	Trọng tâm	Gate
1	30/07–02/08	JS syntax rescue, array/string/matrix/Map/Set/sort	8 template Tầng 1
2	03/08–09/08	Implementation/simulation/hash/sort	Timed set 60 phút
3	10/08–16/08	Stack/queue/heap/binary search	Mock nền tảng 120 phút

Giai đoạn	Ngày	Trọng tâm	Gate
4	17/08–23/08	DFS/BFS/grid/graph/backtracking	Mock graph 120 phút
5	24/08–28/08	Greedy/DP/prefix và PCCP tổng hợp	Không thêm pattern sau 28/08
6	29/08–04/09	Hai mock chính thức, tổng duyệt, sửa lỗi, taper	5 mock tổng cộng

6.1 Khối lượng thực tế

- Ngày thường: 170–220 phút.
- Ngày mock: 120 phút thi + 75–90 phút hậu kiểm.
- Ngày cuối: 60–90 phút, dừng sớm.
- 48 bài chất lượng: nhóm A 29, nhóm B 14, nhóm C 5.
- 5 mock: hai synthetic nền tảng/graph, hai mock chính thức, một tổng duyệt cuối.
- Bài sai quan trọng được review R1/R3/R7; không tính “đã đọc lại” là “đã làm lại”.
- Tracker giữ đúng ngày toán học D+1/D+3/D+7 kể cả khi rơi vào/sau 05/09; các mốc đó là hậu kỳ tùy chọn và không chen vào buổi thi.

6.2 Mỗi tuần bắt buộc có

- một tổng ôn;
- một timed practice hoặc mock;
- một lần viết template không nhìn;
- một lần phân tích Error Log;
- một quyết định bỏ bớt bài nhóm C nếu backlog; P45 là ngoại lệ đã khóa cho Mock 5.

Phần 7 — Khóa học và bộ luyện phù hợp trên Programmers

Thứ tự	Khóa/bộ	Mục đích	Học phần nên chọn	Bỏ/hoãn	Thời gian	Fit 37 ngày
1	Hello, JavaScript: 자바스크립트 입문	Cứu cú pháp JS	biến/kiểu, String, Array, if, loop, function, scope	môi trường, comment, this, closure trong lượt đầu	2h16 toàn khóa; chỉ cần 90–120 phút chọn lọc	Rất phù hợp D1–D4
2	코딩테스트 광탈 방지 A to Z : JavaScript	DSA và code JS có hệ thống	array/object, stack, queue, hash, heap, binary, sort, DFS/BFS, greedy; DP/backtracking nếu cần	linked list, trie và bonus CS trong lượt đầu	4h35 video + bài tập; trả phí	Phù hợp có chọn lọc, không bắt buộc mua
3	코딩테스트 고득점 Kit	Bài chuẩn theo pattern	Các bài thuộc Kit trong tracker; phần còn lại lấy từ kho bài chính thức Programmers	Không làm cả kit theo số lượng	Theo tracker	Một nguồn luyện chính
4	PCCP 기출문제 + 8 lesson riêng	Học phong cách PCCP công khai	P01–P08	Course 19344 chỉ gom một phần, không coi là đủ	8 bài	Bắt buộc
5	PCCP 모의고사 1회	Mock chính thức 4 câu	Mở lần đầu 29/08, đủ 120 phút	Không xem trước	120 + 90 phút review	Bắt buộc
6	PCCP 모의고사 2회	Mock chính thức 4 câu	Mở lần đầu 31/08, đủ 120 phút	Không xem trước	120 + 90 phút review	Bắt buộc
7	PCCP 기출문제 해설 강의 (Python)	Đối chiếu reasoning sau attempt	Chỉ xem bài đã tự làm bằng JS	Không chép Python→JS khi chưa tự code	1h54 toàn khóa	Phù hợp sau attempt, miễn phí

Thứ tự	Khóa/bộ	Mục đích	Học phân nên chọn	Bỏ/hoãn	Thời gian	Fit 37 ngày
8	프로그래머스와 함께하는 PCCP 합격 대비 : 실전 모의고사 해설 강의(Python편)	Tài nguyên mở rộng	Chỉ dùng nếu đã xong lịch chính	Python, trả phí, 15 bài; không tối ưu cho người thi JS	7h28	Không ưu tiên
9	Sách coding test JavaScript	Tra cứu dài hạn	Chương đúng weak pattern	Không đọc tuần tự trong 37 ngày	Tùy chọn	Không cần để bắt đầu

Không tìm thấy khóa video PCCP chuyên biệt bằng JavaScript trong các tài nguyên được rà soát. Vì vậy, tuyến chính là **template JS → bài PCCP/High Score Kit bằng JS → video Python chỉ để đối chiếu reasoning sau attempt**.

Phần 8 — Danh sách 48 bài luyện

8.1 Quy ước

- **A — bắt buộc:** làm theo lịch, review nghiêm túc.
- **B — nên làm:** giữ nếu không có backlog nhóm A.
- **C — còn thời gian:** bỏ trước khi làm hỏng lịch mock.
- **N/A** ở P09–P16 nghĩa là mock không công bố Level, không phải Level 0.
- Bảng CSV đầy đủ chứa `pattern` phụ, `hint rule`, `redo rule`, hidden-risk chi tiết, ngày R1/R3/R7 và cột kết quả: [PCCP Problem Tracker.csv](#).

8.2 Nhóm A — 29 bài bắt buộc

ID	Bài	Level chính thức	Pattern chính	Ngân sách nghĩ
P01	Băng bó	Lv.1	Simulation theo sự kiện	30 phút
P02	Khai thác đầu	Lv.2	BFS/DFS component	50 phút
P05	Trình phát video	Lv.1	Simulation + time parsing	30 phút
P06	Thử thách game xếp hình	Lv.2	Binary search on answer	45 phút
P07	Tìm nguy cơ va chạm	Lv.2	Simulation nhiều route	55 phút
P09	Chữ cái cô lập	N/A	String/implementation	Mock
P10	Đại hội thể thao	N/A	Brute force/backtracking	Mock
P12	Hệ điều hành	N/A	Heap/scheduling	Mock
P13	Robot thực hành	N/A	Simulation	Mock
P14	Đào tạo nhân viên mới	N/A	Heap/greedy	Mock
P15	Mở rộng quán cà phê	N/A	Queue/simulation	Mock
P16	Bản đồ kho báu	N/A	BFS/grid	Mock
P17	Hạn lưu trữ dữ liệu cá nhân	Lv.1	Date parsing/Map	30 phút
P18	Dao công viên	Lv.1	Grid simulation	30 phút
P19	Game gấp thú	Lv.1	Stack + matrix	35 phút
P20	Cuộc đua chạy	Lv.1	Map trạng thái	30 phút

ID	Bài	Level chính thức	Pattern chính	Ngân sách nghĩ
P21	Người chưa hoàn thành	Lv.1	Frequency Map	25 phút
P22	Ponketmon	Lv.1	Set	20 phút
P23	Danh bạ điện thoại	Lv.2	Sort + prefix	35 phút
P24	Trang phục	Lv.2	Frequency Map + counting	35 phút
P25	Số thứ K	Lv.1	Slice + numeric sort	20 phút
P26	Phát triển tính năng	Lv.2	Queue/grouping	Mock
P27	Tiến trình	Lv.2	Queue	35 phút
P28	Cay hơn	Lv.2	Min-heap	Mock
P29	Xuồng cứu sinh	Lv.2	Greedy + two pointer	35 phút
P30	Số mục tiêu	Lv.2	DFS/backtracking	Mock
P31	Đường ngắn nhất bản đồ game	Lv.2	BFS grid	Mock
P32	Sự kiện giảm giá	Lv.2	Sliding window + frequency	Mock
P33	Tổng dãy con liên tiếp	Lv.2	Two pointer	45 phút

8.3 Nhóm B – 14 bài nên làm

ID	Bài	Level chính thức	Pattern chính	Ngân sách nghĩ
P03	Đồng hồ analog	Lv.2	Math/event counting	60 phút
P08	Khôi phục biểu thức	Lv.3	Parsing + hypothesis	60 phút
P11	Quy luật di truyền	N/A	Recursion/math	Mock
P34	Số lớn nhất	Lv.2	Custom comparator	Mock
P35	Xe tải qua cầu	Lv.2	Queue/simulation	Mock
P36	Giá cổ phiếu	Lv.2	Stack	45 phút
P37	Tạo số lớn	Lv.2	Greedy stack	45 phút
P38	Hệ thống đánh chặn	Lv.2	Interval greedy	40 phút
P39	Đô mê tô	Lv.2	Backtracking	Mock
P40	Mạng lưới	Lv.3	DFS/BFS component	Mock
P41	Thoát mê cung	Lv.2	BFS nhiều chặng	50 phút
P42	Biến đổi số	Lv.2	BFS/DP state	45 phút
P43	Tam giác số nguyên	Lv.3	DP 2D	Mock
P44	Đường đến trường	Lv.3	DP grid	50 phút

8.4 Nhóm C – 5 bài khi còn thời gian

ID	Bài	Level chính thức	Pattern chính	Ngân sách nghĩ
P04	Di chuyển xe kéo	Lv.3	Backtracking nhiều tác nhân	60 phút
P45	Bộ điều khiển đĩa	Lv.3	Heap/scheduling	Mock; C nhưng khóa cho Mock 5
P46	Chuyển đổi từ	Lv.3	BFS word graph	60 phút
P47	Kiểm tra nhập cảnh	Lv.3	Binary search on answer	60 phút

ID	Bài	Level chính thức	Pattern chính	Ngân sách nghĩ
P48	Node xa nhất	Lv.3	Graph BFS	55 phút

8.5 Hint, lời giải và redo

Tình huống	Quy tắc
Bài dễ	20–30 phút tự nghĩ
Bài trung bình	40–60 phút tự nghĩ
Trước hint	Ghi yêu cầu, constraints, brute force, complexity và câu “tôi đang kẹt ở...”
Hint cấp 1	Chỉ xem pattern hoặc bottleneck
Hint cấp 2	Xem invariant/pseudocode, không xem code
Lời giải	Chỉ sau khi timebox + hint vẫn chưa triển khai được
Sau lời giải	Đóng tài liệu, code lại từ file trống
Redo	Dùng <code>redo_rule</code> của từng dòng tracker. Các ngày R1/R3/R7 là mốc recall; full recode bắt buộc đúng theo rule, đặc biệt với bài hint/WA/timeout/mock
Mock	Không hint, không tài liệu, không AI trong 120 phút

Hidden-risk cụ thể của từng bài nằm trong tracker; không thay bằng checklist chung.

Phần 9 — Quy trình học một bài

9.1 Quy trình bắt buộc 16 bước

1. Đọc đề và tự viết lại yêu cầu bằng 2–4 câu.
2. Gạch constraints, input/output, tie-break.
3. Viết brute force trước.
4. Tính complexity của brute force.
5. Chỉ ra bottleneck và nhận diện pattern.
6. Bắt đầu timebox tự nghĩ.
7. Viết pseudocode 5–15 dòng.
8. Viết một câu invariant/state meaning.
9. Code rõ ràng, không code golf.
10. Tự tạo test tối thiểu, biên, cực đại và test phá invariant.
11. Submit.
12. Với test fail, thu nhỏ case và phân loại lỗi trước khi sửa.
13. Chỉ xem hint khi đã ghi rõ bottleneck.
14. Nếu xem lời giải, đóng lời giải rồi tự code lại từ file trống.
15. Làm lại sau 1 và 3 ngày.
16. Làm lại sau 7 ngày nếu là nhóm A, từng hint, WA hoặc timeout.

9.2 Khi nào dừng mắc kẹt

- 10–12 phút không viết được brute force/pattern: xem lại constraints, rồi đổi bài nếu đang timed.
- Hết ngân sách 20–30/40–60 phút: ghi bottleneck và dùng hint theo cấp.
- Nhiều lỗi code liên tiếp nhưng không biết invariant: dừng patch, dựng test nhỏ và trace tay.
- Không ngồi nhiều giờ chỉ để bảo vệ “streak không xem lời giải”. Mục tiêu là học có vòng phản hồi.

Phần 10 — Error Log

Mẫu dùng được trong CSV/Google Sheets/Notion: [PCCP Error Log Template.csv](#).

10.1 Phân loại lỗi

- Không hiểu đề.
- Sai thuật toán.
- Sai complexity.
- Sai syntax.
- Sai index.
- Sai boundary.
- Sai state initialization.
- Quên reset state.
- Sort sai.
- Number overflow/precision.
- Queue chậm do `shift()`.
- DFS recursion quá sâu.
- Đọc nhầm input/output.
- Pass sample nhưng fail hidden.

10.2 Hai rule bắt buộc từ lỗi thật

Sai	Đúng	Root cause	Rule
<code>arr[row, col]</code>	<code>arr[row][col]</code>	Toán tử comma không phải truy cập 2D	Matrix luôn dùng hai cặp ngoặc vuông
<code>col < arr.length</code>	<code>col < arr[row].length</code>	Giả định matrix vuông	Số hàng và số cột có thể khác

Một dòng Error Log tốt phải trả lời:

Test nhỏ nào làm lộ lỗi? Vì sao code cũ sai? Quy tắc ngăn nào ngăn lỗi tái diễn? Ngày nào sẽ code lại?

Không ghi “cấu thả” hoặc “quên” làm root cause duy nhất.

Phần 11 — Hidden test checklist

Input và cấu trúc

- ☐ Input rỗng có được phép không?
- ☐ Chỉ có một phần tử.
- ☐ Tất cả phần tử giống nhau.
- ☐ Có duplicate, zero, số âm nếu constraints cho phép.
- ☐ Giá trị cực đại.
- ☐ Matrix không vuông; row/column bằng 1.
- ☐ Điểm bắt đầu bằng điểm kết thúc.
- ☐ Không có đáp án; có nhiều đáp án/tie-break.

JavaScript

- ☐ Numeric sort có `(a, b) => a - b` hoặc comparator đúng?
- ☐ `sort`, `reverse`, `splice` có mutate input ngoài ý muốn?

- ☐ Matrix rows có độc lập hay dùng chung reference?
- ☐ State/closure có reset giữa lượt gọi?
- ☐ Map missing key dùng `?? 0` đúng chỗ?
- ☐ Return đúng kiểu và format?
- ☐ Tổng có vượt `Number.MAX_SAFE_INTEGER`? Nếu dùng `BigInt`, đã không trộn `number`?

Thuật toán

- ☐ Complexity phù hợp constraints?
- ☐ Có loop ẩn như `indexOf/includes` bên trong loop lớn?
- ☐ BFS đánh `visited` lúc enqueue?
- ☐ Queue có `shift()` liên tục?
- ☐ DFS có nguy cơ call stack sâu?
- ☐ Binary search có predicate đơn điệu, cập nhật cận và thoát?
- ☐ `</<=`, inclusive/exclusive đúng?
- ☐ State được reset/undo đúng?
- ☐ Graph directed hay undirected, node 0-based hay 1-based?

Sample pass chỉ cho thấy code qua sample. Trước submit phải có ít nhất một test làm lộ invariant chính.

Phần 12 — Chiến thuật thi 120 phút

12.1 Timeline

Phút	Hành động
0–8	Scan cả 4 bài; ghi constraints, pattern, fit; xếp theo bản thân
8–28	Bài chắc nhất
28–58	Bài thứ hai
58–92	Bài thứ ba khả thi nhất
92–102	Bài còn lại hoặc hoàn thiện bài gần xong
102–116	Audit hidden/efficiency/index/state/Number
116–120	Chạy sample, kiểm return, submit từng câu

12.2 Quy tắc park

Tạm bỏ bài khi:

- 10–12 phút chưa có brute force hoặc pattern;
- chưa có pseudocode/invariant rõ;
- 30–35 phút chưa có code chạy;
- lỗi nổi tiếp nhưng không xác định state nào sai;
- vượt ngân sách và còn bài chưa đọc.

Lưu lại một dòng trước khi chuyển:

“Tôi đã có ____; còn kẹt ở ____; nếu quay lại sẽ thử ____.”

Không tuyệt đối hóa thứ tự câu thành độ khó. Trải nghiệm cộng đồng nhiều năm cũng không nhất quán về điều này; fit với pattern của bản thân quan trọng hơn.

Phần 13 — Mock test

13.1 Trước và trong mock

- Ngồi đủ 120 phút, không chia phiên.
- Không tài liệu, hint, AI hay lời giải.
- Dùng đúng ngôn ngữ JavaScript và môi trường gần giống thi.
- Ghi thời điểm bắt đầu/kết thúc từng bài.
- Ghi số submit và lý do submit lại.
- Không mở hai course mock chính thức trước ngày đã định.
- Chỉ xem lời giải sau khi timer về 0.

13.2 Báo cáo sau mock

Trường	Nội dung
Bộ mock/ngày	
Thứ tự chọn bài	
Thời gian scan	
Bài 1: nghĩ/code/debug	
Bài 2: nghĩ/code/debug	
Bài 3: nghĩ/code/debug	
Bài 4: nghĩ/code/debug	
Số lần submit	
Lỗi lớn nhất	
Pattern không nhận ra	
Template chưa thuộc	
Hidden test bỏ sót	
Quyết định park đúng/sai	
Ba việc sửa trong 72 giờ	

Không chỉ ghi tổng điểm. Nếu hệ thống không hiển thị điểm PCCP chuẩn, theo dõi số bài hoàn chỉnh, số test pass và chất lượng hậu kiểm; không tự quy đổi thành điểm chính thức.

Phần 14 — Dashboard tiến độ

14.1 Dashboard tuần

Chỉ số	Tuần này	Tuần trước	Mục tiêu tuần tới
Tổng bài attempt			
Tự giải hoàn toàn			
Cần hint cấp 1/2			
Xem lời giải			
Đã code lại từ file trống			
Hoàn tất R1/R3/R7 đúng hạn			
Tỉ lệ tự giải theo pattern			
Thời gian trung bình/bài			

Chi số	Tuần này	Tuần trước	Mục tiêu tuần tới
Lỗi syntax			
Lỗi index/boundary			
Lỗi complexity/timeout			
Mock: bài hoàn chỉnh/test pass			
Quyết định park đúng			
Điểm yếu lớn nhất			

14.2 Chỉ số quan trọng hơn số bài

Ưu tiên theo dõi:

1. Tỷ lệ tự code lại không nhìn sau 3 ngày.
2. Số lỗi cùng root cause tái diễn.
3. Thời gian từ ý tưởng đến code chạy.
4. Tỷ lệ nhận diện pattern đúng trong 60 giây.
5. Tỷ lệ mock giữ được 15 phút cuối.

48 bài làm sâu tốt hơn 100 bài chỉ đọc lời giải.

Phần 15 — Cheat sheet cuối

Đọc/in [PCCP Final Cheat Sheet.md](#) trong 48 giờ cuối. Bản này được giới hạn theo bốn “trang” nội dung:

1. Complexity, array/sort/Map/Set/matrix.
2. Stack/queue/BFS/DFS/graph.
3. Binary search/heap/prefix/window/DP/Number.
4. Hidden test và timeline 120 phút.

Ngày 04/09 chỉ:

- viết lại 8 template Tầng 1;
- đọc Error Log;
- đọc cheat sheet;
- xác nhận giờ thi/thiết bị/hướng dẫn;
- dừng sau 60–90 phút và ngủ đủ.

Nguồn và giới hạn

Danh mục đầy đủ, loại nguồn, nội dung được hỗ trợ và ngày truy cập nằm trong [sources.md](#). Tổng cộng: **37 nguồn nên và 48 trang bài tập chính thức, 85 URL tham chiếu.**

Các giới hạn chưa thể xác minh chính thức:

1. Không có bảng trọng số từng câu/công thức partial score công khai.
2. Không có phiên bản Node.js chung được công bố cho mọi lượt thi.
3. Ngày 05/09/2026 không xuất hiện trong lịch kỳ thi thường lệ công khai khi kiểm tra.
4. Tám bài mock không có Level chính thức.

Khi thông tin lượt thi riêng mâu thuẫn với tài liệu này, hướng dẫn chính thức trong thư mời/pre-test của chính lượt thi được ưu tiên.